INDIAN INSTITUTE OF INFORMATION TECHNOLOGY VADODARA

CS545 – SOFTWARE ENGINEERING

PROJECT / LAB REPORT

QuizInGo – Online Quiz System

Submitted by

Sundram Tripathi [20251651091]

Abhinav kumar [20251651004]

Ankit kumar[20251651022]

Akshay Kumar[20251651012]

Programme: MCA | Year: [2nd year]

Submitted to

Rishabh Gupta

Academic Session: [2026-27]

[29/10/2026]

CERTIFICATE

This is to certify that the report titled “QuizInGo – Online Quiz System” has been prepared by the following student(s) as part of the CS545 – Software Engineering course requirements.

Sundram Tripathi [20251651091]

Abhinav kumar [20251651004]

Ankit kumar[20251651022]

Akshay Kumar[20251651012]

The report describes the project work and experiments documented by the student(s).

Faculty signature: _____

Name: Rishabh Gupta

Date: _____

DECLARATION

I/We declare that this report titled "QuizInGo – Online Quiz System" has been prepared for the CS545 – Software Engineering course. The diagrams, descriptions, screenshots and test results included in the final report should reflect the project and work actually completed.

Name(s): Sundram Tripathi

Student signature(s): _____

Name(s): Abhinav Kumar Chandravanshi

Student signature(s): _____

Name(s): Ankit Kumar

Student signature(s): _____

Name(s): Akshay kumar

Student signature(s): _____

Date: _____

ACKNOWLEDGEMENT

I would like to thank the course faculty, Rishabh Gupta, for guidance and the opportunity to study software engineering concepts through the QuizInGo project. I also acknowledge the contributions of my teammates, where applicable, and the tools and documentation used during development.

TABLE OF CONTENTS

[In Microsoft Word: References → Table of Contents → Automatic Table. Update the table after editing.]

- Experiment Index
- Chapter 1: Introduction and Product Overview
- Chapter 2: System Context and Architecture
- Chapter 3: Software Requirements Specification (SRS)
- Chapter 4: Software Process Model
- Chapter 5: Use Case Diagram
- Chapter 6: Data Flow Diagram (DFD)
- Chapter 7: UML Class Diagram
- Chapter 8: Sequence Diagrams
- Chapter 9: Modular Design
- Chapter 10: Implementation Overview
- Chapter 11: Functional Test Case Design
- Chapter 12: Security, Reliability and Operations
- Chapter 13: Limitations and Future Enhancements
- Chapter 14: Conclusion
- Appendix A: Screenshots and Evidence
- Appendix B: Glossary
- References

EXPERIMENT INDEX

Experiment	Title	Evidence to include	Status
1	Software Requirements Specification (SRS)	SRS sections and requirements table	Complete
2	Selecting a Software Process Model	Model choice, rationale, lifecycle diagram	Complete
3	Use Case Diagram	QuizInGo use-case diagram and explanation	Complete
4	Data Flow Diagram (DFD)	Context diagram and level-1 DFD if required	Complete
5	UML Class Diagram	Class diagram matching actual implementation	Complete
6	Sequence Diagrams	Login, quiz creation, quiz attempt flows	Complete
7	[Only if assigned]	[Add official handout output]	Complete
8	Modular Design of a Software System	Module breakdown and dependency diagram	Complete
9	Functional Test Case Design	Test cases with actual observed results	Complete

CHAPTER 1: INTRODUCTION AND PRODUCT OVERVIEW

1.1 Introduction

QuizInGo is a web-based online quiz application developed to simplify quiz-related activities. The project aims to provide a digital platform where users can interact with quizzes through a web interface.

The application focuses on organizing quiz activities and making them more convenient than a completely manual process. The system's actual capabilities are determined by the modules implemented in the current project version.

1.2 Problem Statement

Managing quizzes manually can involve preparing questions, distributing them to participants, checking answers, calculating scores, and maintaining results. These activities may require additional time and effort.

QuizInGo aims to address these challenges through an online quiz platform. It provides a foundation for managing quiz-related activities in an organized manner.

1.3 Objectives

The main objectives of QuizInGo are:

1. To develop a web-based platform for quiz-related activities.
2. To provide an organized interface for users.
3. To simplify quiz management and participation where supported.
4. To reduce manual evaluation effort where automatic evaluation is implemented.
5. To apply Software Engineering principles during development and testing.

1.4 Project Scope

The scope of QuizInGo includes the functionality implemented in the current version of the application. Potential areas include user authentication, quiz management, quiz participation, and result handling.

Only features that are present in the source code and verified through testing will be considered completed project functionality. Features that are not implemented will be listed as limitations or future enhancements.

1.5 Intended Users

The intended users depend on the roles supported by the application.

- **Students/Participants:** May participate in available quizzes and view results if these functions are implemented.
- **Teachers/Quiz Creators:** May create or manage quizzes if the application provides these permissions.
- **Administrator:** Included only if an administrative role is implemented in the current system.

CHAPTER 2: SYSTEM CONTEXT AND ARCHITECTURE

2.1 System Context

QuizInGo is a web-based application through which users interact with quiz-related functionality using a web browser. The frontend provides the user interface, while the backend handles application requests and business logic. Data is stored through the database configured for the application.

The system context consists of the application users, the QuizInGo frontend, the backend services, and the configured data storage.

2.2 Technology Stack

The project documentation describes the following technologies. Confirm each item against the current source code before final submission.

Layer	Technology	purpose
Frontend	React with Vite	Builds the user interface
Backend	Node.js and Express	Handles server-side requests
Database	MongoDB	Stores application data, if configured
Authentication	JWT	Supports token-based authentication,

2.3 Architecture Diagram

The high-level architecture of QuizInGo can be represented as:

User → Web Browser → React Frontend → Express Backend/API → MongoDB Database

The frontend sends requests to the backend. The backend processes requests and interacts with the configured database when data access is required. Responses are returned to the frontend and displayed to the user.

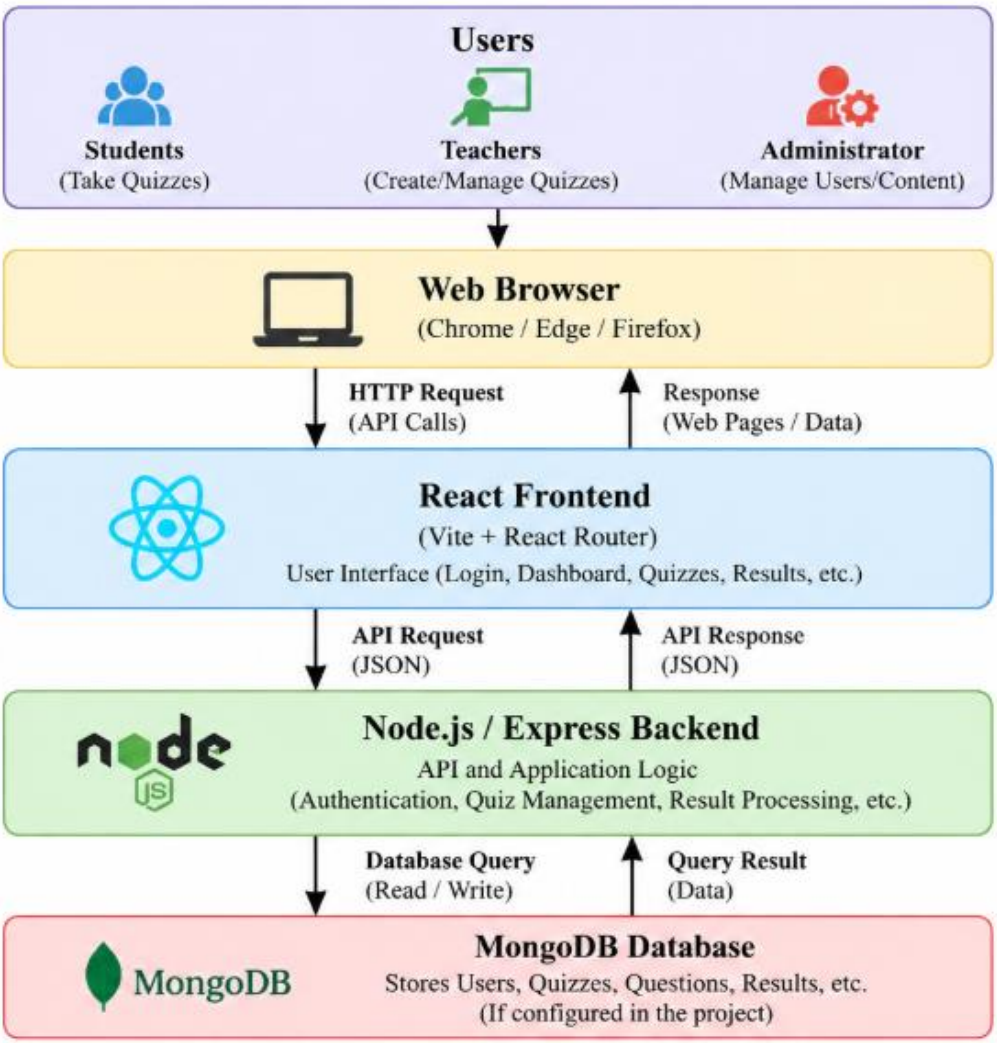


Figure 2.1: High-Level Architecture of QuizInGo

2.4 Main Components

1. Frontend: Provides the pages and interface through which users interact with the application.

2. Routing: Supports navigation between application pages.

3. Backend/API: Processes client requests and implements server-side operations.

4. Database: Stores application data when a database connection is configured and working.

5. Authentication: Controls access to protected functionality if authentication and authorization are implemented.

2.5 Conclusion

The architecture separates the user interface from server-side processing and data storage. This separation helps organize the application into components that can be developed, tested, and maintained independently.

CHAPTER 3: SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

3.1 Purpose

The Software Requirements Specification (SRS) defines the functional and non-functional requirements of QuizInGo. It describes the expected system behaviour and provides a basis for software design, implementation, and testing.

3.2 Functional Requirements

3.2 Functional Requirements

ID	Requirement	Priority	Verification
FR-01	Users can register and log in, if implemented.	High	Test registration and login
FR-02	Authorized users can create and manage quizzes, if supported.	High	Test quiz management
FR-03	Users can add or manage quiz questions, if supported.	High	Test question management
FR-04	Participants can attempt and submit quizzes, if implemented.	High	Submit a test quiz
FR-05	The system can calculate and display quiz results, if implemented.	High	Verify the result after submission

3.3 Non-functional Requirements

- **Usability:** The interface should be understandable and easy to navigate.
- **Performance:** Common operations should respond within a reasonable time.
- **Security:** Protected operations should be accessible only to authorized users.
- **Reliability:** The application should handle valid operations consistently.
- **Maintainability:** The code should be organized to support future changes.

3.4 Assumptions and Constraints

1. Users need a compatible web browser to access the application.
2. Some features may require an active backend service.

3. Data-related functionality depends on the database configuration.
4. Authentication and access control depend on their actual implementation.
5. The requirements listed above must be verified against the current project before final submission.

CHAPTER 4: SOFTWARE PROCESS MODEL

4.1 Selected Model

After evaluating the software process models described in Experiment 2, the **Agile Process Model (Scrum)** has been selected for the QuizInGo Online Quiz System.

Agile is suitable for projects where requirements may change during development, early delivery of working software is required, and continuous feedback from users is important.

4.2 Reason for Selection

The requirements of an online quiz system may evolve based on feedback from students and instructors. Features such as the user interface, timer handling, question navigation, grading, and reporting may require improvements during development.

The Agile model supports this through:

1. **Iterative Development:** The system can be developed in small increments called sprints.
2. **Early Delivery:** Working versions can be made available for testing at an early stage.
3. **Continuous Feedback:** Students and instructors can provide feedback to improve usability.
4. **Flexibility:** Changes in requirements can be considered in subsequent sprints.
5. **Early Defect Detection:** Frequent testing helps identify problems during development.

4.3 Process Stages

The Agile (Scrum) process can be represented through the following stages:

1. **Product Backlog:** Prepare a prioritized list of features and requirements.
2. **Sprint Planning:** Select the features to be developed in the next sprint.
3. **Requirement Analysis:** Refine the requirements for the selected features.
4. **Design:** Plan the interface, architecture, and data organization.
5. **Implementation:** Develop the selected functionality.
6. **Testing:** Test the implemented features and correct identified defects.

7. **Sprint Review:** Demonstrate the working increment and collect feedback.
8. **Increment Release:** Deliver a working version when appropriate.
9. **Continuous Improvement:** Update the backlog and repeat the cycle.

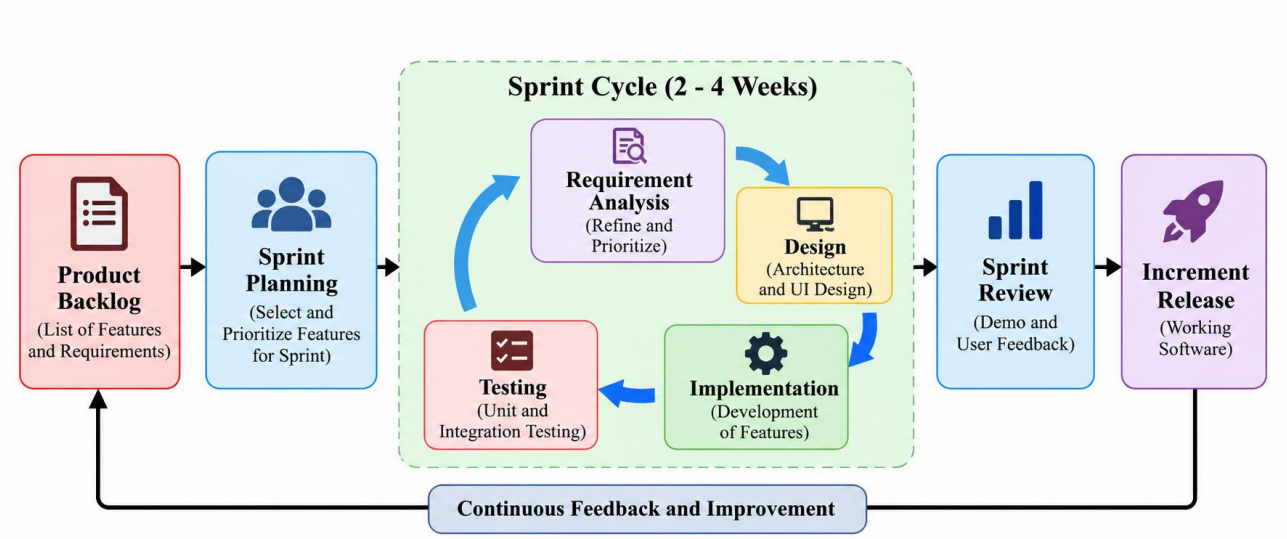


Figure 4.1: Agile (Scrum) Process Model for QuizInGo

4.4 Justification Against Other Process Models

Waterfall Model: It is less flexible when requirements change after the initial planning stage.

Incremental Model: It supports delivery in parts, but Agile places additional emphasis on regular feedback and adapting priorities.

Prototype Model: It is useful for exploring interface ideas, but prototyping alone does not define the complete development and release process.

Spiral Model: It emphasizes risk analysis and may involve more overhead than needed for a typical educational quiz application.

4.5 Conclusion

The Agile (Scrum) process model is selected for QuizInGo because it supports iterative development, early delivery, continuous user feedback, and adaptation to changing requirements. These characteristics make it a suitable choice for the Online Quiz System described in Experiment 2.

CHAPTER 5: USE CASE DIAGRAM

5.1 Actors

Actors are the external users who interact with the QuizInGo system to perform specific activities. The main actors shown in the use case diagram are:

1. Student: The student interacts with the system to log in, view available quizzes, attempt quizzes, submit answers, view results, and check quiz history, where these features are supported.

2. Teacher: The teacher interacts with the system to log in, create quizzes, manage questions, configure quiz settings, publish quizzes, view submissions, and view result analytics, where these features are supported.

5.2 Use Cases

A use case represents a function or activity that an actor can perform through the system. The main use cases shown in the QuizInGo diagram are:

Student Use Cases:

- Register and log in to the system.
- View available quizzes.
- Attempt a quiz and submit answers.
- View quiz results.
- View quiz history.

Teacher Use Cases:

- Log in to the system.
- Create and manage quizzes.
- Manage quiz questions.
- Configure quiz timer and settings.
- Publish or unpublish quizzes.
- View student submissions.
- View result analytics.

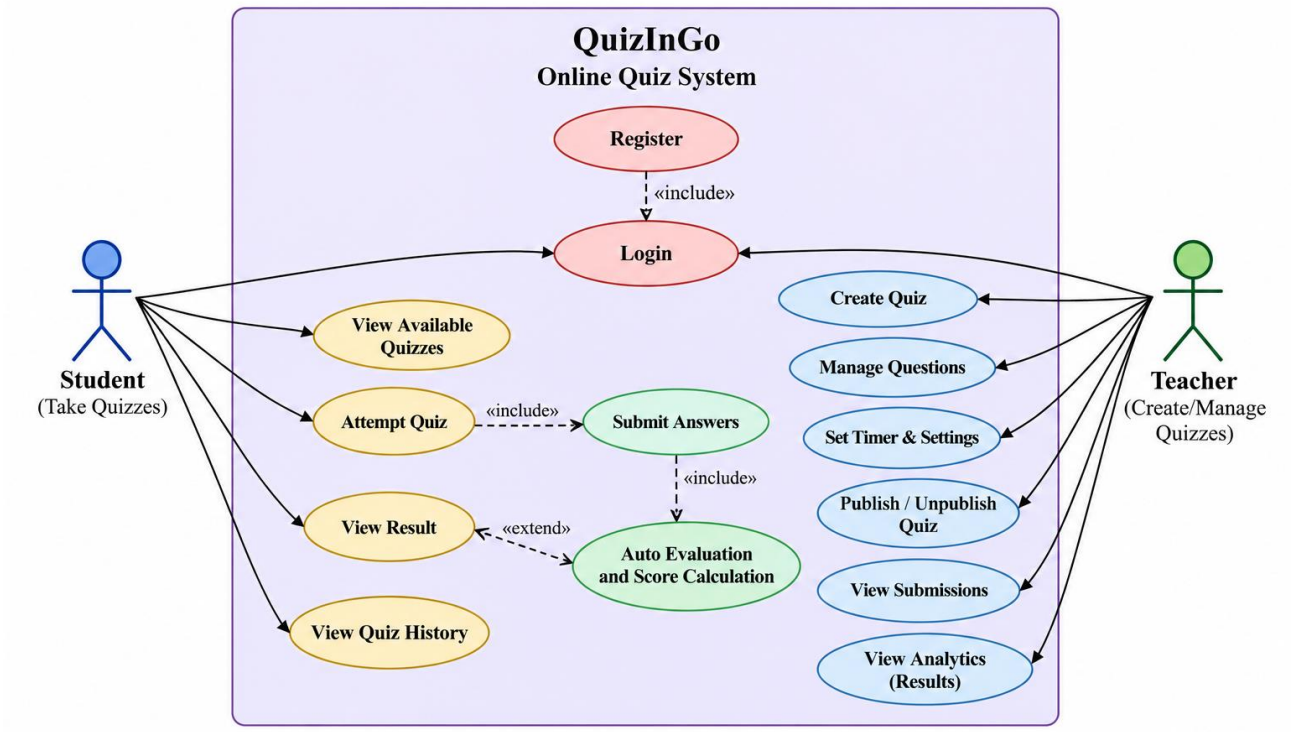
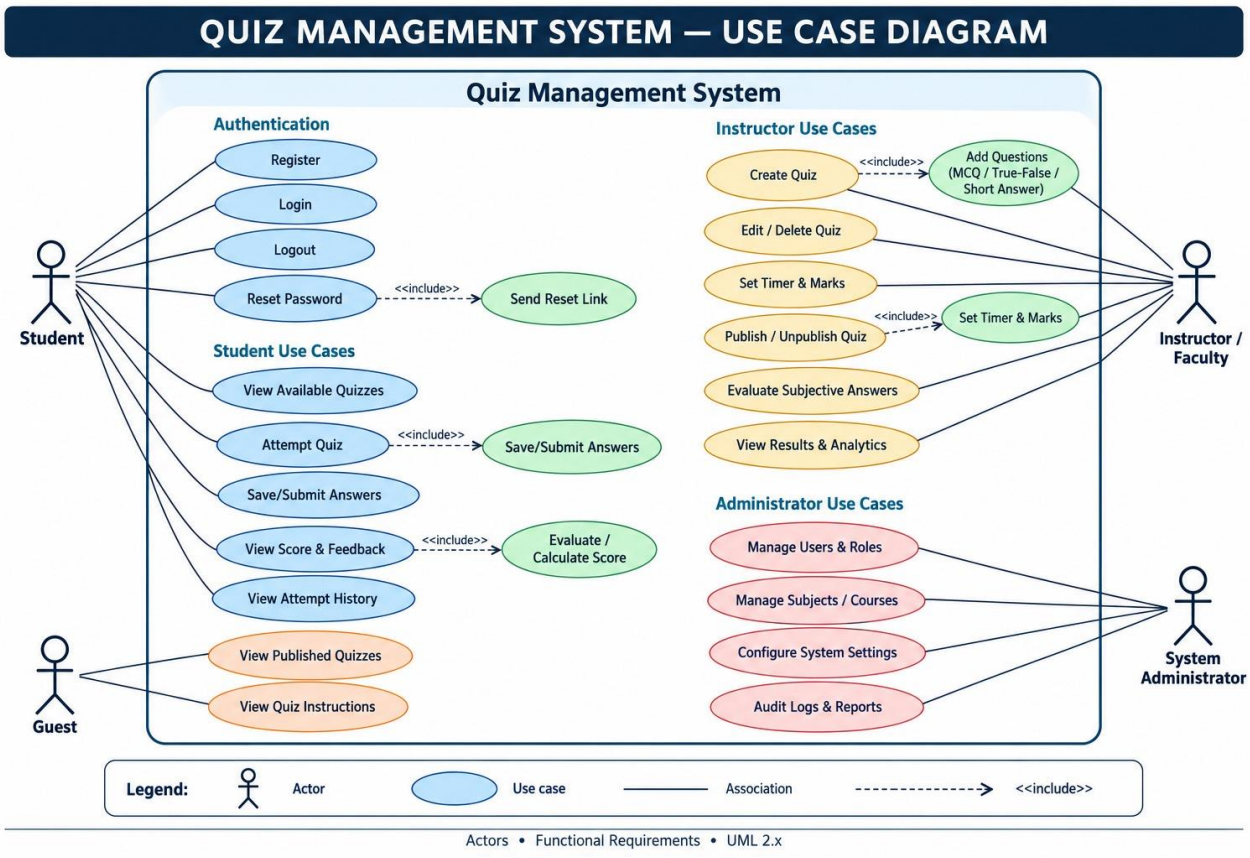


Figure 5.1: Use Case Diagram of QuizInGo

2nd Plan:-



5.3 Use Case Description

Use Case 1: Student Login

Field	Description
Use Case	Student Login
Primary Actor	Student
Purpose	To access the QuizInGo system
Preconditions	The student has an account and the application is accessible.
Main Flow	1. Student opens the login page. 2. Student enters credentials. 3. The system validates the credentials. 4. The system grants access if the credentials are valid.
Alternate Flow	If the credentials are invalid, the system displays an error message.
Postconditions	The student is logged in successfully, or access is denied.

Use Case 2: Create Quiz

Field	Description
Use Case	Create Quiz
Primary Actor	Teacher
Purpose	To create a quiz for students
Preconditions	The teacher is logged in and has permission to create quizzes.
Main Flow	1. Teacher opens the quiz creation page. 2. Teacher enters quiz details. 3. Teacher adds questions. 4. Teacher saves the quiz.
Alternate Flow	If required information is missing, the system asks the teacher to correct it.
Postconditions	The quiz is saved if the operation succeeds.

Use Case 3: Attempt Quiz

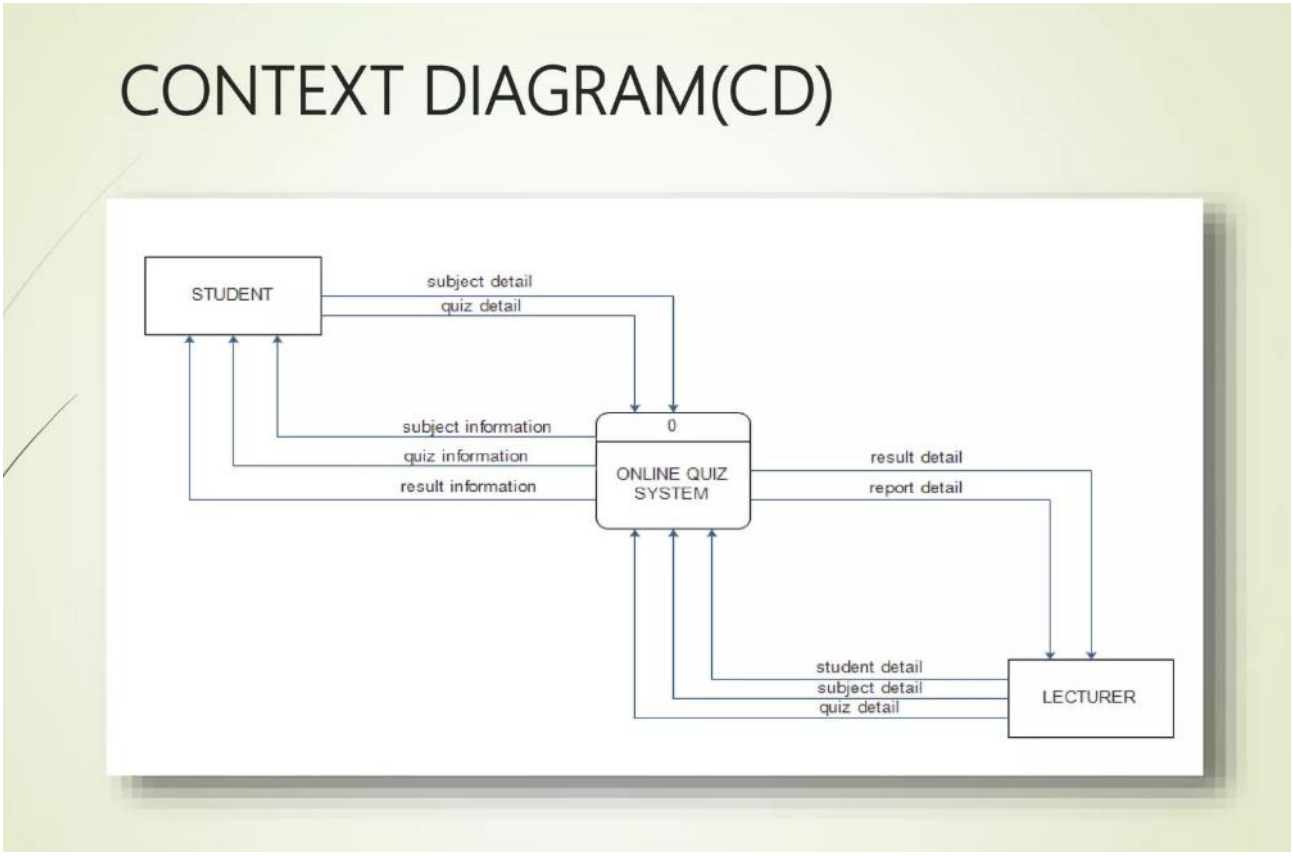
Field	Description
Use Case	Attempt Quiz
Primary Actor	Student
Purpose	To answer and submit a quiz
Preconditions	The student is logged in and an available quiz exists.
Main Flow	1. Student opens an available quiz. 2. Student reads the questions. 3. Student selects answers. 4. Student submits the quiz.
Alternate Flow	If submission fails, the system should provide an appropriate error or retry option, if implemented.
Postconditions	The submitted answers are recorded if the submission succeeds.

These use case descriptions explain the expected interaction between users and the system. The final report should reflect the actual behaviour implemented in QuizInGo.

CHAPTER 6: DATA FLOW DIAGRAM (DFD)

6.1 Context Diagram

The Context Level DFD represents QuizInGo as a single system and shows its interaction with external entities. The main external entities are Student and Teacher. Students send login details, quiz requests, and submitted answers, while receiving available quizzes, questions, and results. Teachers send quiz creation and management requests and may receive submission information and result analytics, depending on the implemented features.

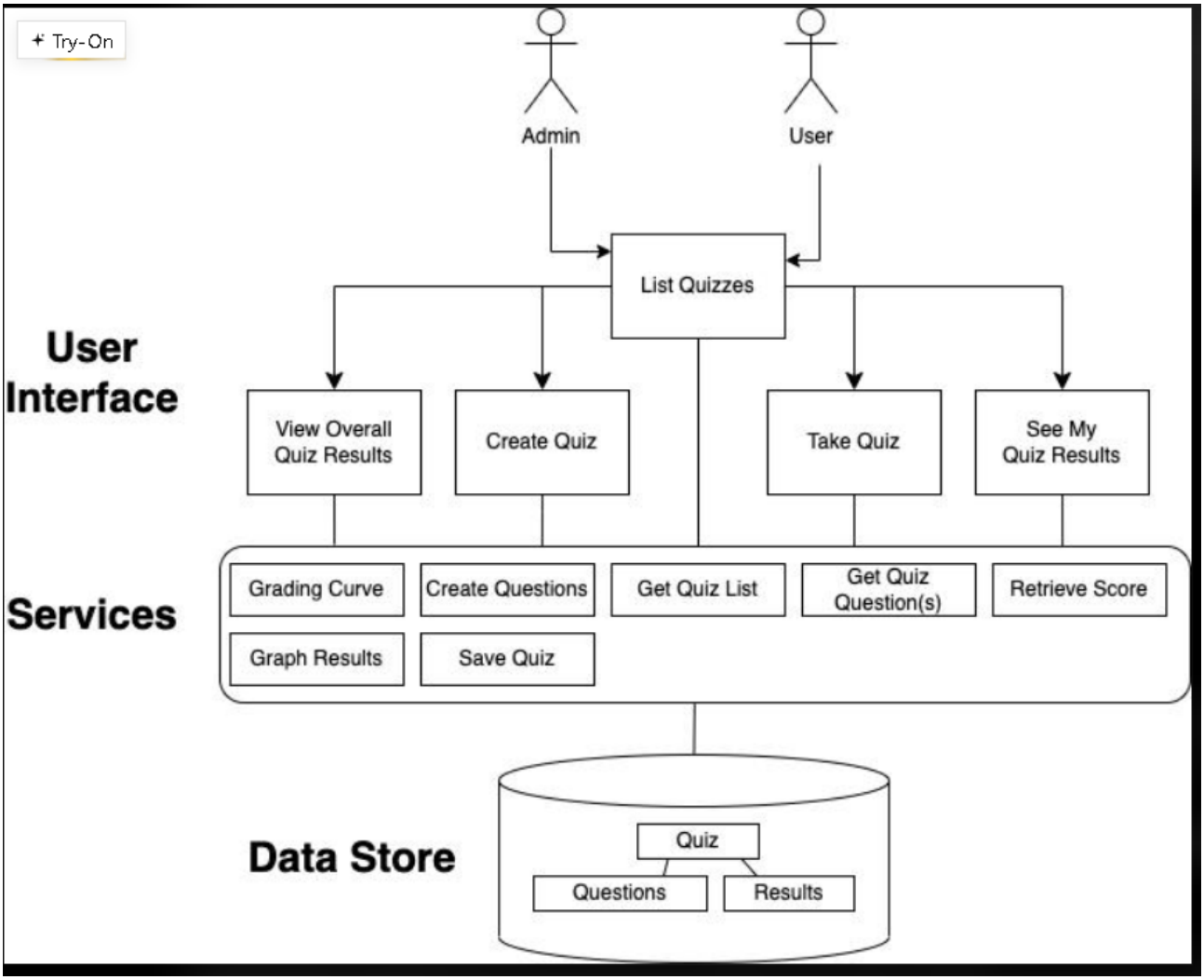


6.2 Level-1 DFD

The Level-1 DFD divides QuizInGo into smaller processes to explain how data moves through the system.

- **Process 1.0 – User Management:** Handles user-related operations, if implemented.
- **Process 2.0 – Quiz Management:** Manages quiz details and questions, if supported.
- **Process 3.0 – Quiz Attempt:** Handles quiz requests and answer submissions.
- **Process 4.0 – Result Processing:** Processes submitted answers and results, if implemented.

The data stores may include user information, quiz information, submissions, and results. These stores must correspond to the actual database structure used by the project.



6.3 Data Stores and Processes

The main processes describe the functional activities of the QuizInGo system. Data stores represent the information retained by the application, such as user details, quiz details, submitted answers, and results.

The arrows in the DFD indicate the direction of data transfer between external entities, processes, and data stores. The final diagram should be updated to match the actual modules and database collections implemented in QuizInGo.

6.1 Context Diagram (Level 0 DFD)

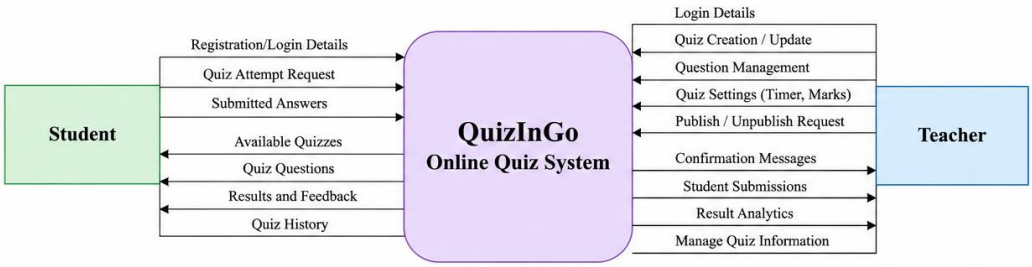


Figure 6.1: Context Diagram of QuizInGo

6.2 Level-1 Data Flow Diagram

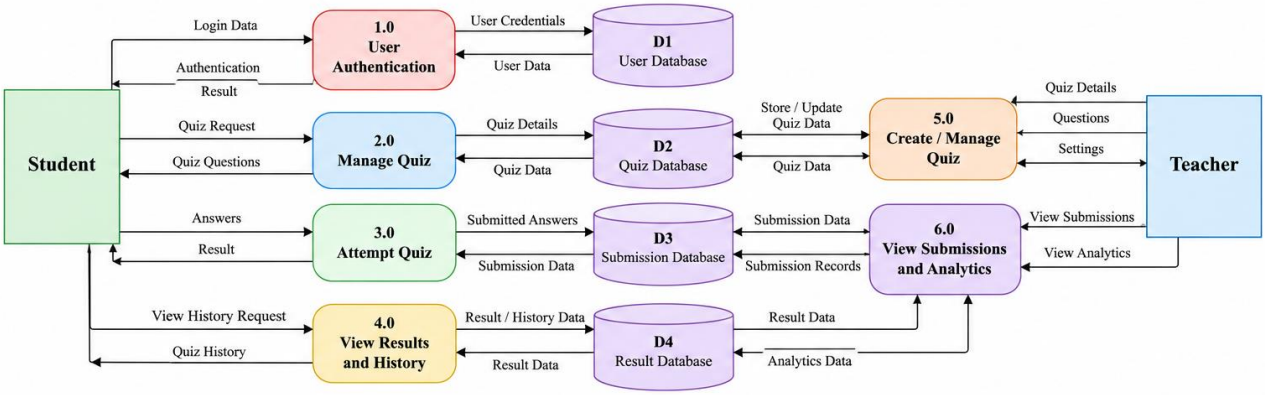


Figure 6.2: Level-1 Data Flow Diagram of QuizInGo

CHAPTER 7: UML CLASS DIAGRAM

7.1 Main Classes

The QuizInGo project uses several models to represent and manage the data required by the online quiz system. The main models identified in the project source code are User, Quiz, Question, Attempt, Classroom, and OtpChallenge.

1. **User:** Stores user-related information and supports different user roles, including students and teachers.
2. **Quiz:** Represents a quiz and stores its details, including questions associated with the quiz.
3. **Question:** Represents question-bank data used for quiz-related activities.
4. **Attempt:** Records a user's quiz attempt, submitted answers, and score-related information.
5. **Classroom:** Represents classroom-related information and student membership.
6. **OtpChallenge:** Represents OTP challenge information used in authentication-related workflows.

These models represent the main data entities identified in the source code. Their exact attributes and behaviour depend on their implementation.

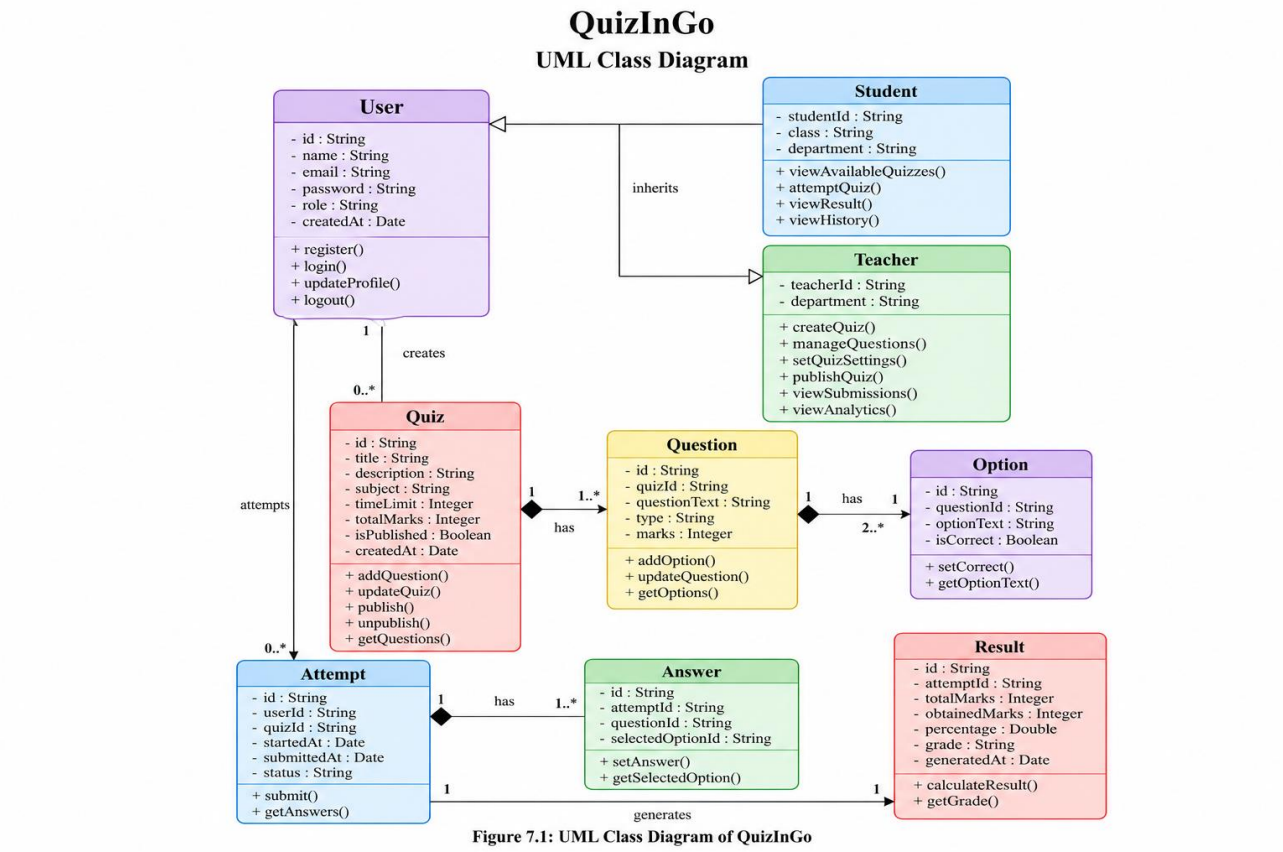


Figure 7.1: UML Class Diagram of QuizInGo

7.2 Relationships

The relationships between the main QuizInGo models can be described as follows:

- **User and Quiz:** User and Quiz models represent user information and quiz information, respectively. The association should follow the creator or ownership fields defined in the actual schema.
- **Quiz and Question:** Quiz-related question data is used to represent the questions associated with quizzes. The exact relationship depends on the implemented schema.
- **User and Attempt:** Attempt records are associated with the user taking a quiz.
- **Quiz and Attempt:** Attempt records identify the quiz being attempted.
- **Classroom and User:** Classroom-related data supports classroom membership and student association.
- **OtpChallenge and User:** OTP challenge information supports authentication-related workflows, according to the implementation.

The project does not use a separate Result model in the inspected model list. Score-related information is stored in the Attempt model.

7.3 Class Description Table

Class / Model	Purpose
User	Stores user and role information
Quiz	Stores quiz details and related questions
Question	Represents question-bank data
Attempt	Records quiz attempts, answers and score-related information
Classroom	Manages classroom information and membership
OtpChallenge	Stores OTP challenge information

CHAPTER 8: SEQUENCE DIAGRAMS

Create one sequence diagram per important scenario. Lifelines and messages must match the actual QuizInGo implementation.

8.1 User Login

The User Login Sequence Diagram illustrates the interaction between a user, the frontend, the backend server, and the database during the login process in QuizInGo.

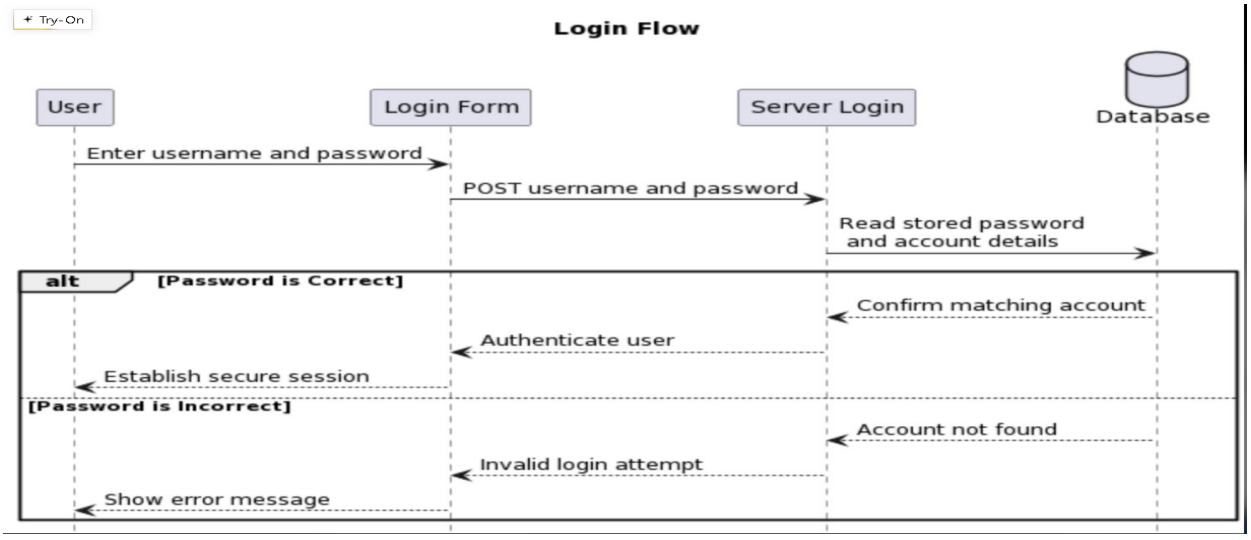
The process starts when the user opens the login page and enters their email and password. The frontend sends the login request to the backend server. The backend validates the provided credentials using the application's authentication logic. After validation, the backend returns a response to the frontend, which displays the appropriate result to the user.

Steps Involved

1. The user opens the login page.
2. The system displays the login form.
3. The user enters their email and password.
4. The frontend sends the login request to the backend.
5. The backend validates the credentials.
6. The backend receives the required authentication information.
7. The backend sends a success or error response to the frontend.
8. The frontend displays the login result to the user.

Result

The sequence diagram describes the expected communication between the user interface and backend during login. Successful authentication allows the user to access the application, while invalid credentials should result in an appropriate error response.



8.2 Teacher Creates/Publishes a Quiz

The Teacher Creates/Publishes a Quiz Sequence Diagram illustrates the interaction between the teacher, frontend, backend server, and database during quiz creation and publication in QuizInGo.

The process begins when the teacher opens the quiz creation page and enters the quiz details. The teacher adds questions, answer options, and the correct answers. The frontend sends the information to the backend, which validates the request and stores the quiz data.

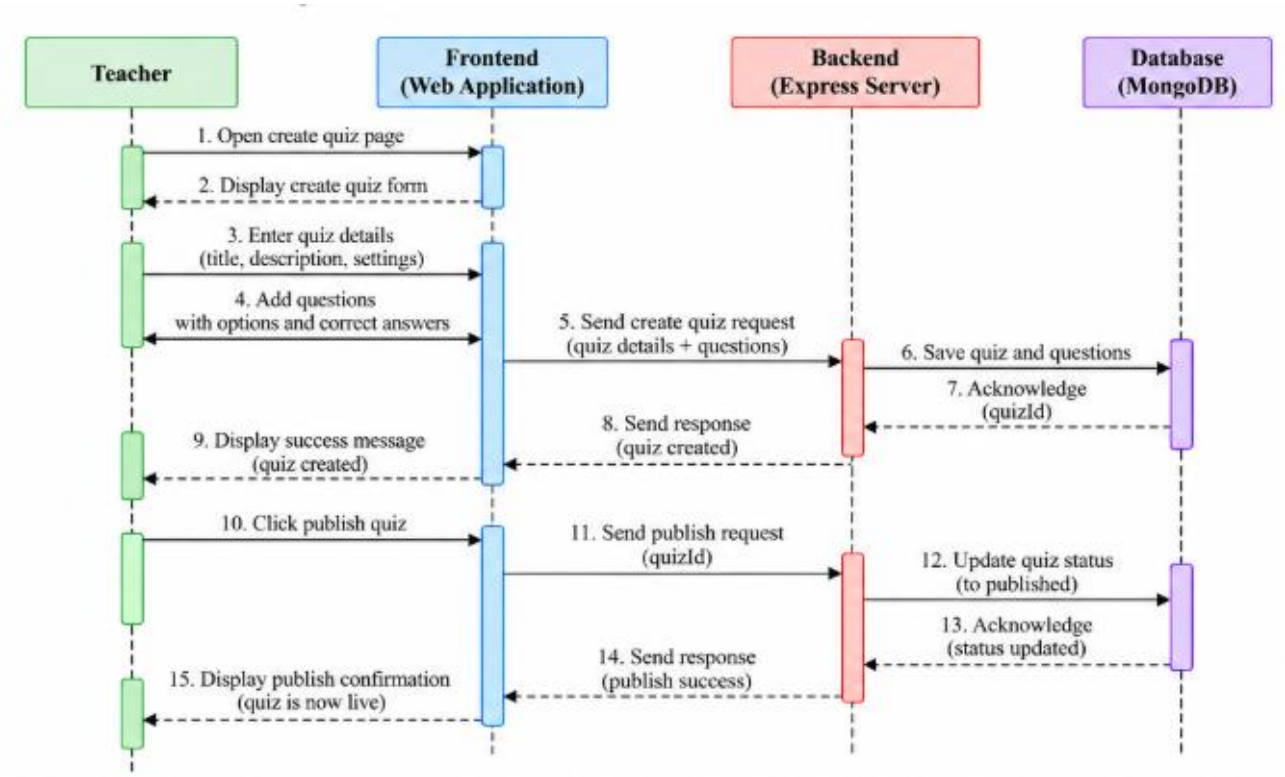
After creating the quiz, the teacher can publish it if the application supports this functionality. The backend processes the publish request and updates the quiz status. Finally, the frontend displays a confirmation message to the teacher.

Steps Involved

1. The teacher opens the quiz creation page.
2. The system displays the quiz creation form.
3. The teacher enters the quiz details.
4. The teacher adds questions and answer options.
5. The frontend sends the quiz creation request to the backend.
6. The backend validates the request and saves the quiz data.
7. The backend returns a response to the frontend.
8. The teacher requests publication of the quiz, if supported.
9. The backend updates the publication status, if implemented.
10. The frontend displays the operation status to the teacher.

Result

The sequence diagram describes the expected communication during quiz creation and publication. Successful completion depends on the implemented functionality and the backend response.



8.3 Student Attempts and Submits a Quiz

The Student Attempts and Submits a Quiz Sequence Diagram illustrates the interaction between the student, frontend, backend server, and database during quiz participation in QuizInGo.

The process begins when the student views the available quizzes and selects one. The frontend requests the quiz details and questions from the backend. After the quiz information is received, the frontend displays the questions to the student.

The student answers the questions and submits the quiz. The backend processes the submitted answers, evaluates them according to the quiz rules, and records the attempt. The system then returns the available result information to the frontend for display.

Steps Involved

1. The student views the available quizzes.
2. The frontend requests the quiz list from the backend.
3. The student selects a quiz.
4. The frontend requests the selected quiz details.
5. The backend retrieves and returns the quiz information.
6. The student answers the questions.
7. The student submits the quiz.
8. The frontend sends the submitted answers to the backend.
9. The backend evaluates the answers and calculates the score, if supported.

10. The system records the attempt and related result information.
11. The backend returns a response to the frontend.
12. The frontend displays the submission status or result to the student.

Result

The sequence diagram represents the expected flow of quiz participation and submission. The exact evaluation, storage, and result-display behaviour should match the implemented QuizInGo functionality.

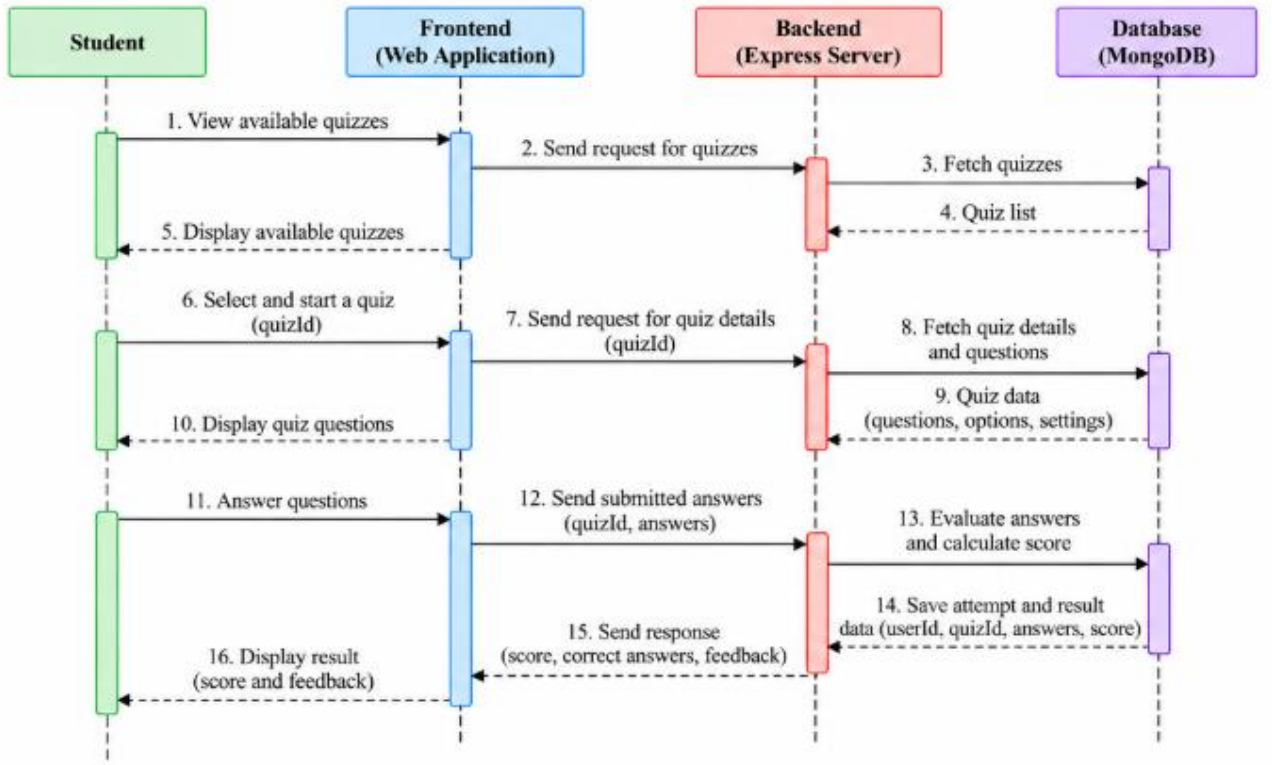


Figure 8.3: Sequence Diagram for Student Attempts and Submits a Quiz

9: Modular Design of the Online Quiz System

Oct 26, 2026 · @Abhinav

Aim: *To divide the Online Quiz System into modules, define interfaces and justify cohesion and coupling.*

The design splits the system into 13 modules in four layers, so that each requirement in the SRS is owned by exactly one module. The requirements below are the ones the modules are traced to.

SRS requirements covered

ID	Requirement	Type

ID	Requirement	Type
FR-01	Users register and log in as admin, teacher or student	Functional
FR-02	Teachers manage a question bank with categories and difficulty	Functional
FR-03	Teachers create, schedule and publish quizzes	Functional
FR-04	Students attempt a quiz with a countdown timer and automatic submission	Functional
FR-05	Objective answers are graded automatically, with optional negative marking	Functional
FR-06	Students see scores and per-question feedback after submission	Functional
FR-07	Teachers view class performance and question statistics	Functional
FR-08	The system flags suspicious behaviour during an attempt	Functional
FR-09	Users receive notifications about quizzes and results	Functional
NFR-01	Passwords are hashed and every request is authenticated	Security
NFR-02	Sensitive actions are recorded in an audit trail	Security
NFR-03	The timer is enforced on the server, not the browser	Reliability
NFR-04	Support 500 simultaneous quiz attempts	Performance

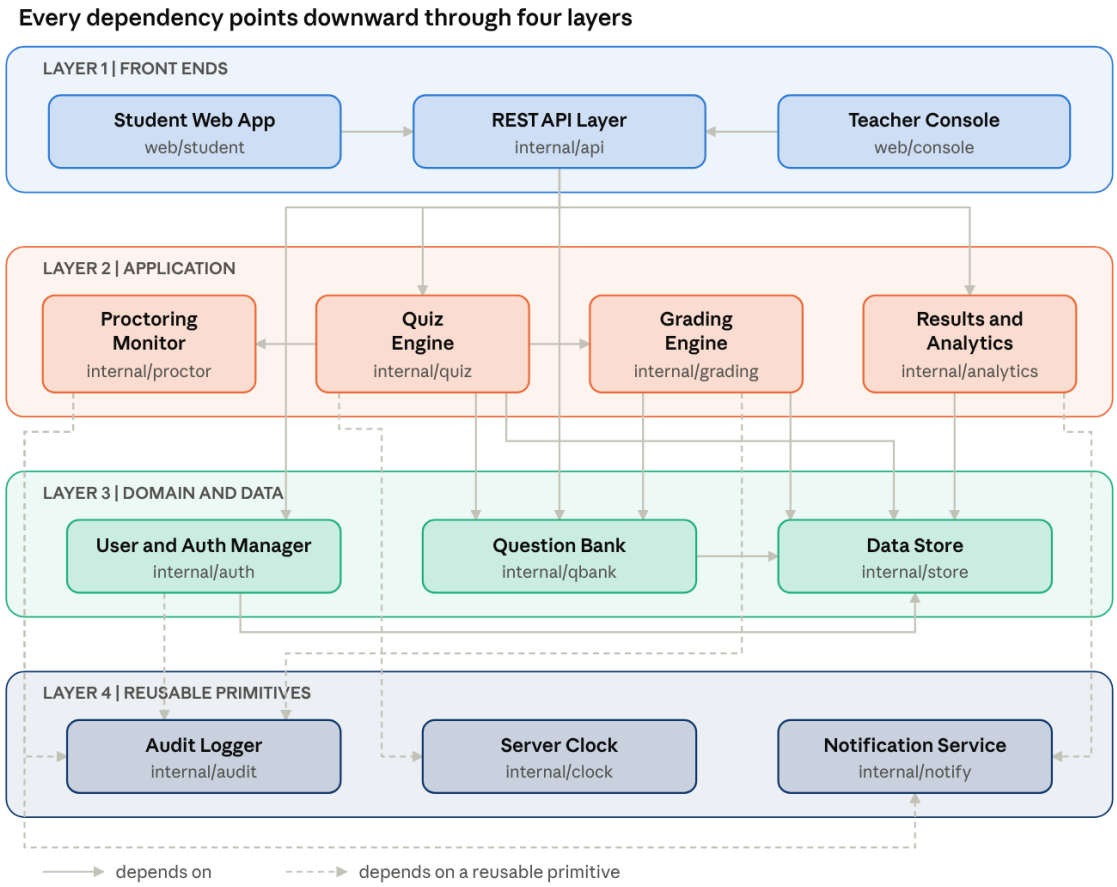
9.1 Modules

The 13 modules are listed from the user-facing layer down to the reusable primitives, and each one carries the SRS requirements it implements.

Module	Package	Main responsibility	SRS
Student Web App	web/student	Browse quizzes, attempt with a live timer, view results	FR-04, FR-06
Teacher Console	web/console	Manage questions, quizzes, users and reports	FR-02, FR-03, FR-07
REST API Layer	internal/api	HTTP endpoints, request validation and wiring of modules	NFR-01
Quiz Engine	internal/quiz	Create, schedule and publish quizzes; start, save and submit attempts	FR-03, FR-04
Grading Engine	internal/grading	Auto-grade answers, apply negative marking, queue manual review	FR-05

Module	Package	Main responsibility	SRS
Results and Analytics	internal/analytics	Scores, ranks, question statistics and class reports	FR-06, FR-07
Proctoring Monitor	internal/proctor	Detect tab switches, duplicate sessions and time anomalies	FR-08
User and Auth Manager	internal/auth	Registration, login, roles and session tokens	FR-01, NFR-01
Question Bank	internal/qbank	Store questions, options, answer keys, categories and difficulty	FR-02
Data Store	internal/store	Repository for users, questions, quizzes, attempts and results	NFR-04
Notification Service	internal/notify	Email and in-app notices for quizzes, results and alerts	FR-09
Audit Logger	internal/audit	Append-only record of logins, grade changes and flags	NFR-02
Server Clock	internal/clock	Authoritative time for deadlines, timers and auto-submit	NFR-03

Architecture overview



layered architecture · 13 modules, 21 dependencies

Fig. 8.1 Modular design diagram of the Online Quiz System (an arrow means depends on).

9.2 Interfaces

Six interfaces carry the main collaborations; modules talk only through these, never through each other's data.

Calling → Providing module	Interface	Input	Output
REST API Layer → Quiz Engine	StartAttempt, SaveAnswer, SubmitAttempt, GetAttemptState	quizId, studentId, questionId, answer	attemptId and remaining seconds, or error
Quiz Engine → Grading Engine	GradeAttempt	attemptId, answer sheet	total score, per-question marks
Quiz Engine, Grading Engine → Question Bank	GetQuestions, GetAnswerKey	quizId or question ids	questions without keys, or answer keys
Quiz Engine → Proctoring Monitor	ReportEvent	attemptId, event type, timestamp	status OK, WARN or FLAGGED

Calling → Providing module	Interface	Input	Output
Quiz Engine → Server Clock	Now, DeadlineFor	start time, duration	current time, attempt deadline
Data-owning modules → Data Store	Repository: Save, FindById, Query	entity, id or filter	stored entity or result rows

9.3 Cohesion and coupling

Every module has one job: the Grading Engine only turns answers into marks, the Question Bank only stores questions and keys, and the Server Clock only tells time. Coupling is kept low because the Grading Engine receives a plain answer sheet and never imports the Quiz Engine, and because only the Data Store touches the database. The graph has 21 connections among 13 modules, so the average coupling is 21/13 = 1.62. Instability is I = Fout / (Fin + Fout), where Fin counts modules that depend on a module and Fout counts modules it depends on.

Module	Fin	Fout	I
Student Web App	0	1	1.00
Teacher Console	0	1	1.00
Quiz Engine	1	5	0.83
Grading Engine	1	3	0.75
REST API Layer	2	4	0.67
Results and Analytics	1	2	0.67
Proctoring Monitor	1	2	0.67
User and Auth Manager	1	2	0.67
Question Bank	3	1	0.25
Data Store	5	0	0.00
Audit Logger	3	0	0.00
Notification Service	2	0	0.00
Server Clock	1	0	0.00

The pattern is the intended one: the front ends are unstable (I = 1.00) because they may change often, while the Data Store, Audit Logger, Notification Service and Server Clock are stable (I = 0.00) because many modules rely on them. All dependencies point downward, so there are no cycles.

9.4 Result

In this experiment we divided the Online Quiz System into 13 modules in four layers, traced each module to the SRS requirements it implements, defined six interfaces and checked cohesion and coupling.

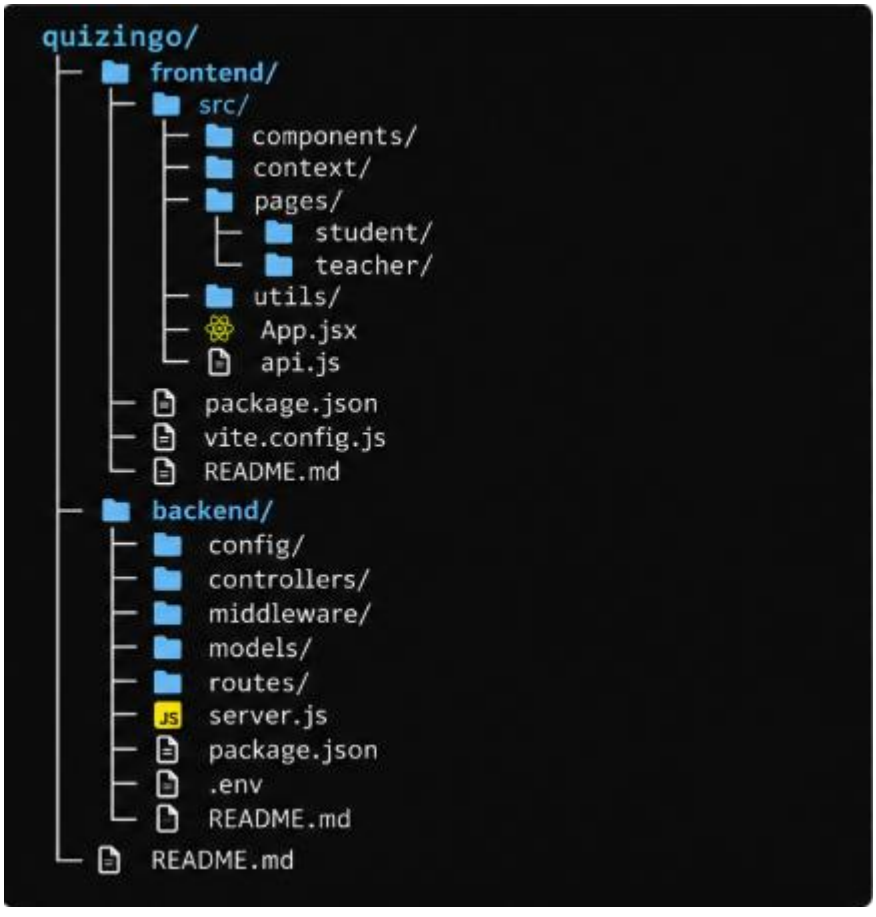
CHAPTER 10: IMPLEMENTATION OVERVIEW

CHAPTER 10: IMPLEMENTATION OVERVIEW

10.1 Project Structure

QuizInGo is implemented as a full-stack web application. The frontend and backend are maintained in separate directories to organize the user interface and server-side functionality.

The main project structure is shown below:



The frontend directory contains the React application, reusable components, pages, and API integration. The backend directory contains server configuration, controllers, middleware, database models, and API routes.

10.2 Important Components

1. Frontend: React and Vite are used to develop and run the web interface. React Router supports navigation between application pages.

2. Authentication: The project includes login and registration pages and an authentication context. The backend uses JWT-related functionality for authentication and protected API routes.

3. Quiz Management: The teacher-side interface includes quiz creation and question-management components.

4. Student Module: The student-side interface includes quiz attempts, dashboards, and result-related pages.

5. Backend API: Node.js and Express handle server-side requests through routes and controllers.

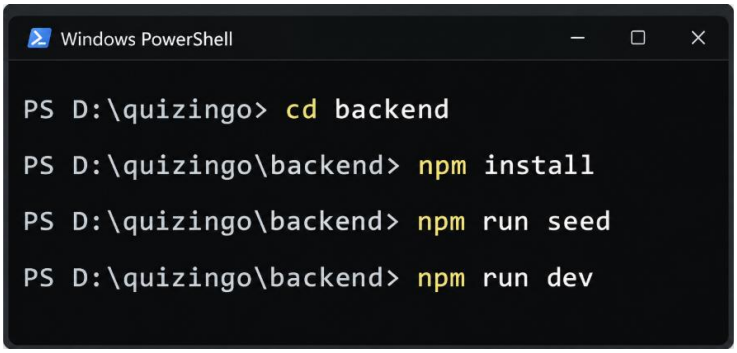
6. Data Models: Mongoose models represent application data, including users, quizzes, questions, attempts, classrooms, and OTP challenges.

7. Database Configuration: The backend includes a database configuration module for connecting to MongoDB. Actual database connectivity depends on the environment configuration.

10.3 Run Instructions

The following commands are documented in the project README. Run them from the project directory in separate terminals.

Backend:



The README specifies the backend address as http://localhost:5000.

Frontend:



The README specifies the frontend address as http://localhost:5173.

The frontend and backend should be configured to communicate with each other. Environment variables must be configured locally as required by the project. Secret values and database credentials must not be included in the report.

10.4 Implementation Summary

QuizInGo is developed as a full-stack web application that separates the frontend interface from backend processing. The frontend provides the user interface, while the backend manages API requests, authentication-related operations, and application data.

The project uses React with Vite for the frontend and Node.js with Express for backend services. The application includes data models for users, quizzes, questions, quiz attempts, classrooms, and OTP challenges.

This modular structure helps organize the application code and supports easier maintenance and future improvements. The implementation can be evaluated by running the application, checking its main features, and recording the observed results during functional testing.

The actual working status of each feature should be confirmed through testing before it is reported as successfully implemented.

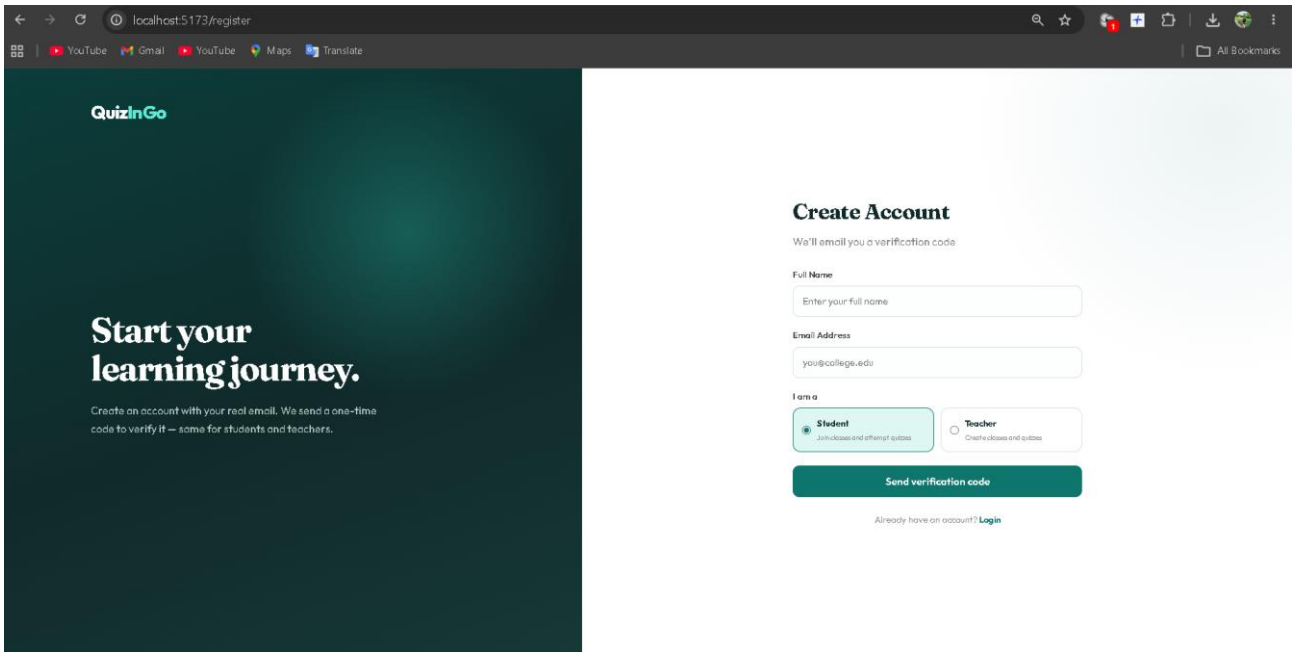
CHAPTER 11: FUNCTIONAL TEST CASE DESIGN

CHAPTER 11: FUNCTIONAL TEST CASE DESIGN

11.1 Introduction

Functional testing is performed to verify whether the functions of QuizInGo behave according to their requirements. Test cases describe the inputs, actions, expected results, and observed results for specific application features.

The test cases below cover important workflows such as user login, quiz creation, quiz participation, and submission. The actual results and status must be recorded after executing each test case.

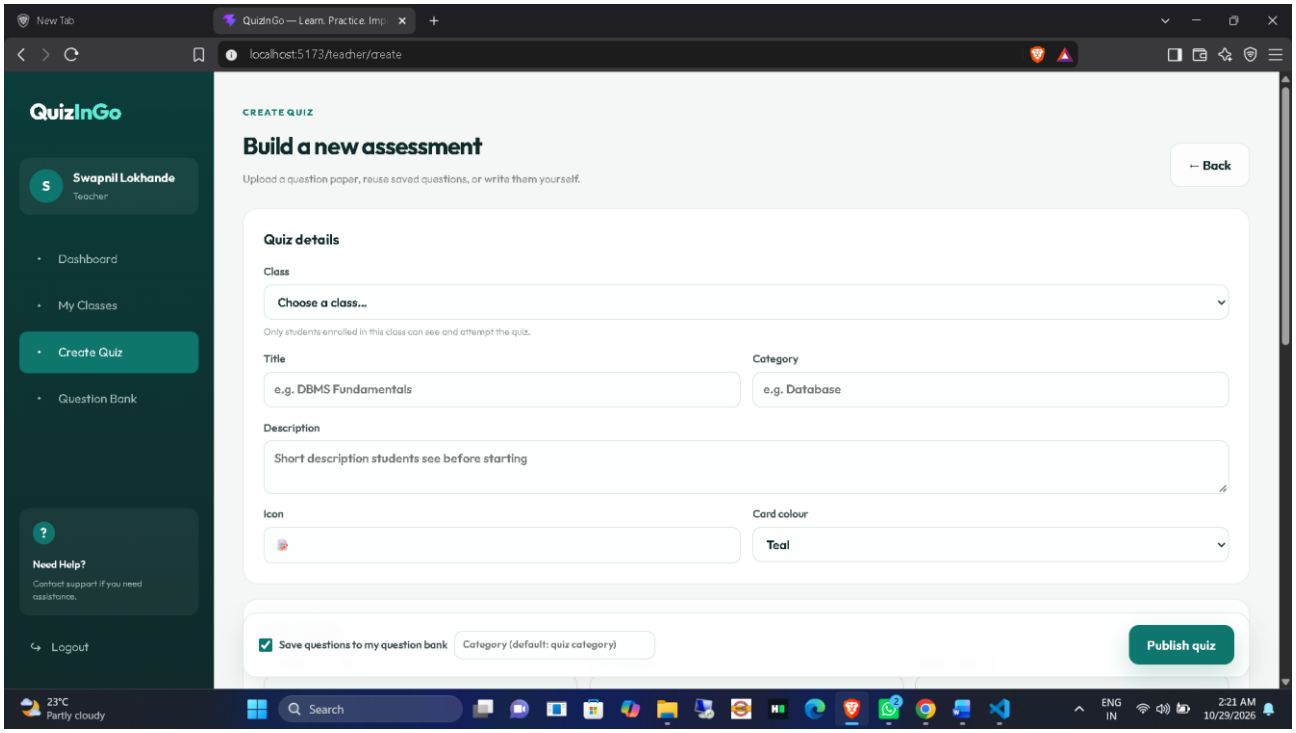


11.2 Functional Test Cases

TC ID	Test Case	Actual Result	Status
TC-01	User Login	Dashboard opened successfully	Pass
TC-02	Invalid OTP Verification	Not tested yet	Not Run
TC-03	Valid OTP Login	OTP accepted; dashboard opened	Pass
TC-04	View Available Quizzes	Quiz list and details loaded	Pass
TC-	Attempt and Submit Quiz	Quiz submitted; result/score displayed	Pass

05			
TC-06	**View Result / Attempt History**	**Previous attempt and result displayed again**	**Pass**
TC-07	**Teacher Creates a Quiz**	**Quiz saved and appeared in quiz list**	**Pass**
TC-08	**Add Questions**	**Question/options saved and remained visible after reopening**	**Pass**
TC-09	**Required Field Validation**	**Not tested yet**	**Not Run**

Only execute test cases for features supported by the current implementation. Remove or revise cases for features that are not implemented.



11.3 Testing Summary

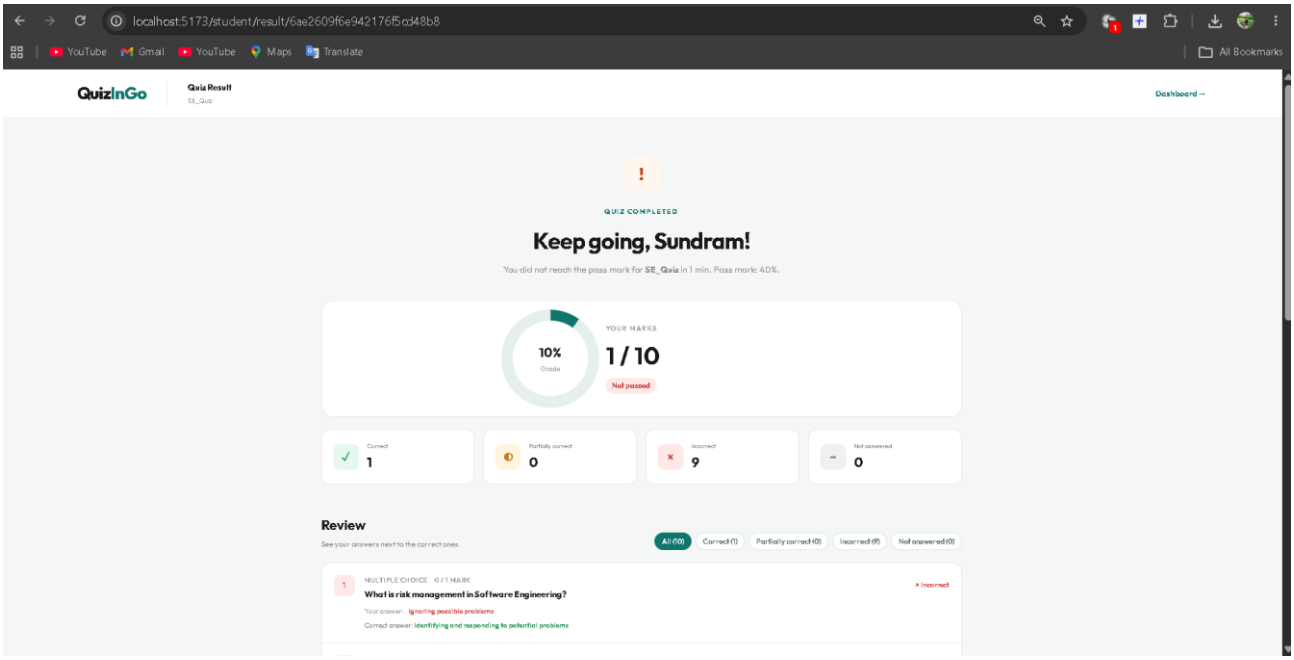
Functional testing was performed on the QuizInGo application to verify its major features, including user login, quiz listing, quiz attempts, result viewing, quiz creation, and question management. The application was tested using the existing MongoDB data without deleting the database or running the seed script.

A total of nine test cases were considered. Seven test cases were executed successfully, while two test cases—Invalid OTP Verification and Required Field Validation—remain untested.

The executed tests confirmed that users could log in, view available quizzes, attempt and submit quizzes, view their results, create quizzes, and add questions. The final testing status is summarized below.

Test Status	Number of Test Cases
Passed	**7**
Failed	**0**
Not Run	**2**
Total	**9**

Overall, the tested features of QuizInGo behaved as expected. Invalid OTP verification and required-field validation must be tested before concluding that all functional test cases have passed.



CHAPTER 12: SECURITY, RELIABILITY AND OPERATIONS

CHAPTER 12: SECURITY, RELIABILITY AND OPERATIONS

12.1 Security

QuizInGo uses authentication to control access to the application. Users verify their identity through an OTP-based login process. JSON Web Tokens (JWT) are used by the backend to support authenticated requests. The backend uses routes, controllers, and middleware to process requests and apply access control.

The application separates student and teacher functionality. Students can view quizzes, attempt quizzes, and check their results, while teachers can create quizzes and manage questions.

Security considerations:

- OTP verification is used during login.
- JWT supports authenticated communication with the backend.
- Backend routes and middleware help protect restricted operations.
- User input should be validated to reduce invalid or unexpected data.
- OTPs and authentication tokens should be handled securely in a production deployment.

12.2 Reliability

QuizInGo is designed to provide the main functions required for online quiz management and participation. During functional testing, the tested features worked as expected, including login, viewing quizzes, submitting quiz attempts, viewing results, creating quizzes, and adding questions.

The application uses MongoDB to store application data. The backend communicates with the database to retrieve and save information. Reliability depends on database availability, correct request processing, and appropriate error handling.

Reliability considerations:

- Verify that the backend and MongoDB connection are available before testing.
- Validate quiz data before saving it.
- Handle invalid requests and database errors appropriately.
- Preserve existing database records during development and testing.
- Perform regular backups before making significant database changes.

12.3 Operations and Deployment

QuizInGo has a frontend and a backend that run as separate applications during local development. The frontend provides the user interface, while the backend exposes API endpoints and connects to MongoDB.

The development environment can be started using the following commands.

Backend:

cd backend

npm install

npm run dev

Frontend (in a separate terminal):

cd frontend

npm install

npm run dev

The README specifies the local frontend address as http://localhost:5173 and the backend address as http://localhost:5000. The MongoDB connection must be configured correctly for database operations to work.

The seed command is not required for ordinary testing and should not be run when preserving existing database data is important. Before deployment, environment variables, authentication security, database access, and error handling should be reviewed.

12.4 Summary

QuizInGo includes authentication, separate student and teacher functionality, and MongoDB-backed data storage. Functional testing confirmed that the tested application features worked as expected. Further verification of invalid OTP handling and required-field validation is still necessary. Additional security checks, backups, and production configuration are recommended before public deployment.

CHAPTER 13: LIMITATIONS AND FUTURE ENHANCEMENTS

CHAPTER 13: LIMITATIONS AND FUTURE ENHANCEMENTS

13.1 Limitations

Although QuizInGo provides the main features of an online quiz system, some limitations remain:

1. **OTP-Based Authentication:** The application uses OTP-based login. Its security depends on proper OTP generation, verification, and handling.
2. **Internet and Server Dependency:** Users need a working network connection and an available backend server to use the application.
3. **Database Dependency:** Quiz, user, question, and attempt data depend on MongoDB availability and correct database configuration.
4. **Limited Testing:** Functional testing has been performed on the main features, but invalid OTP verification and required-field validation are still pending.
5. **Production Readiness:** Additional security checks, error handling, backup procedures, and deployment configuration may be required before public release.

6. **Future Scalability:** The application may require performance optimization and additional infrastructure if the number of users and simultaneous quiz attempts increases.

13.2 Future Enhancements

The following improvements can be considered in future versions of QuizInGo:

1. **Enhanced Security:** Improve OTP handling, authentication controls, input validation, and protection against unauthorized access.
2. **Comprehensive Testing:** Test invalid OTPs, missing required fields, incorrect inputs, network failures, and other edge cases.
3. **Performance Optimization:** Optimize database queries and API responses to support more users and larger quizzes.
4. **Advanced Analytics:** Add dashboards showing quiz performance, student progress, and question-wise statistics.
5. **Timer and Auto-Submission:** Improve timed quiz functionality and handle automatic submission when the time limit expires.
6. **Improved User Experience:** Make the interface more responsive and accessible across mobile phones, tablets, and desktop devices.
7. **Deployment and Backup:** Introduce a reliable production deployment process, database backups, and recovery procedures.

13.3 Conclusion

QuizInGo provides a foundation for online quiz creation, participation, and result viewing. The project demonstrates the integration of a frontend, backend API, authentication, and MongoDB data storage. Addressing the identified limitations and implementing the proposed enhancements can improve the application's security, reliability, usability, and scalability.

CHAPTER 14: CONCLUSION

CHAPTER 14: CONCLUSION

14.1 Conclusion

QuizInGo is an online quiz management system developed to make quiz creation, participation, and result viewing easier. The project provides separate functionality for students and teachers, allowing teachers to create quizzes and add questions while students can view available quizzes, attempt them, submit answers, and check their results.

The project uses React and Vite for the frontend, Node.js and Express.js for the backend, and MongoDB for data storage. OTP-based authentication is used for user login, and JWT supports authenticated requests.

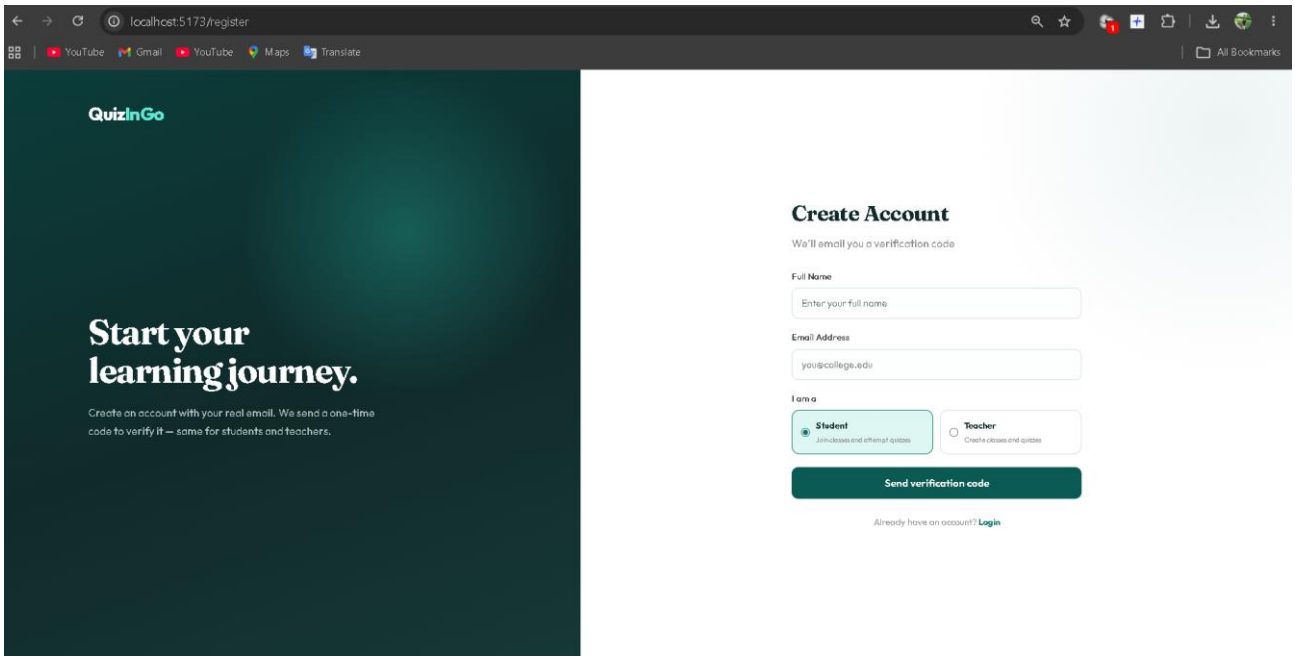
Functional testing was performed using the existing MongoDB data without deleting database records. The tested features included user login, viewing quizzes, attempting and submitting quizzes, viewing results, creating quizzes, and adding questions. These tests worked as expected. Invalid OTP verification and required-field validation remain to be tested.

Through this project, practical knowledge was gained in frontend development, backend API implementation, database integration, authentication, software design, and functional testing.

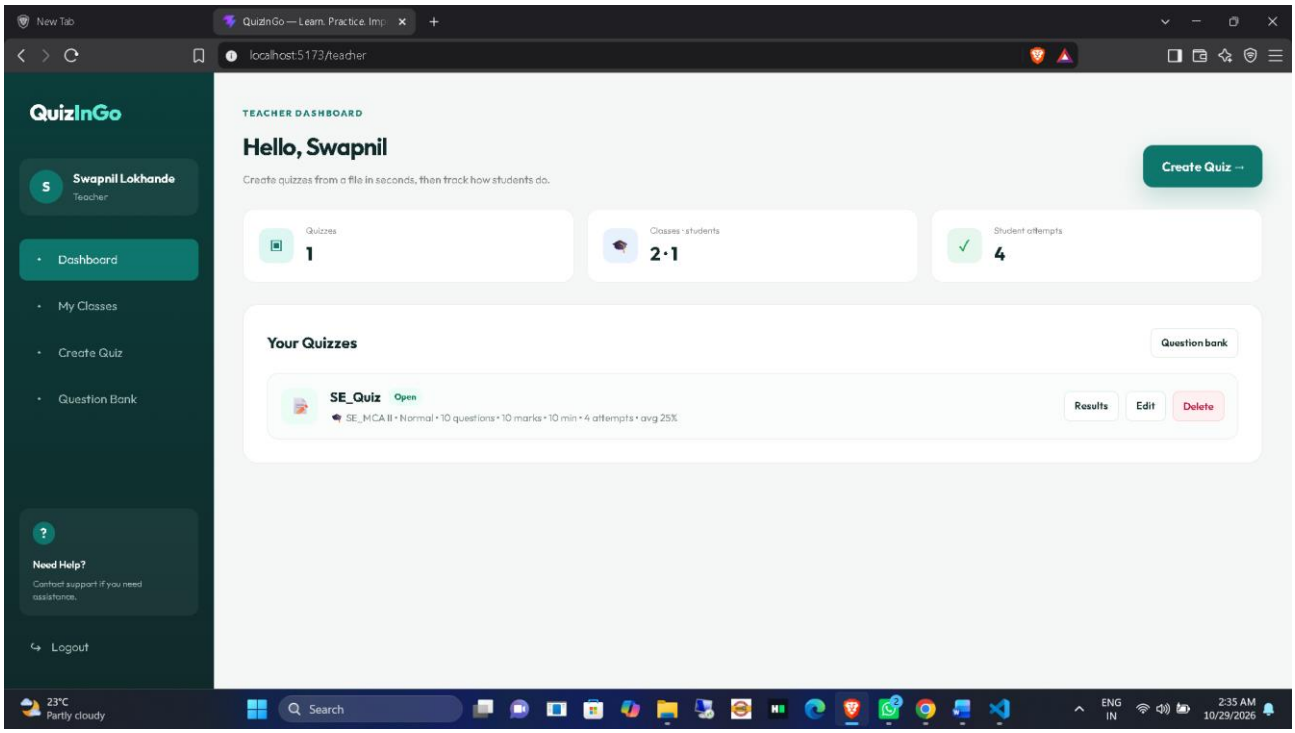
In conclusion, QuizInGo provides a foundation for an online quiz platform. Further improvements in security, validation, performance, testing, and deployment can make the system more reliable and suitable for wider use.

APPENDIX A: SCREENSHOTS AND EVIDENCE

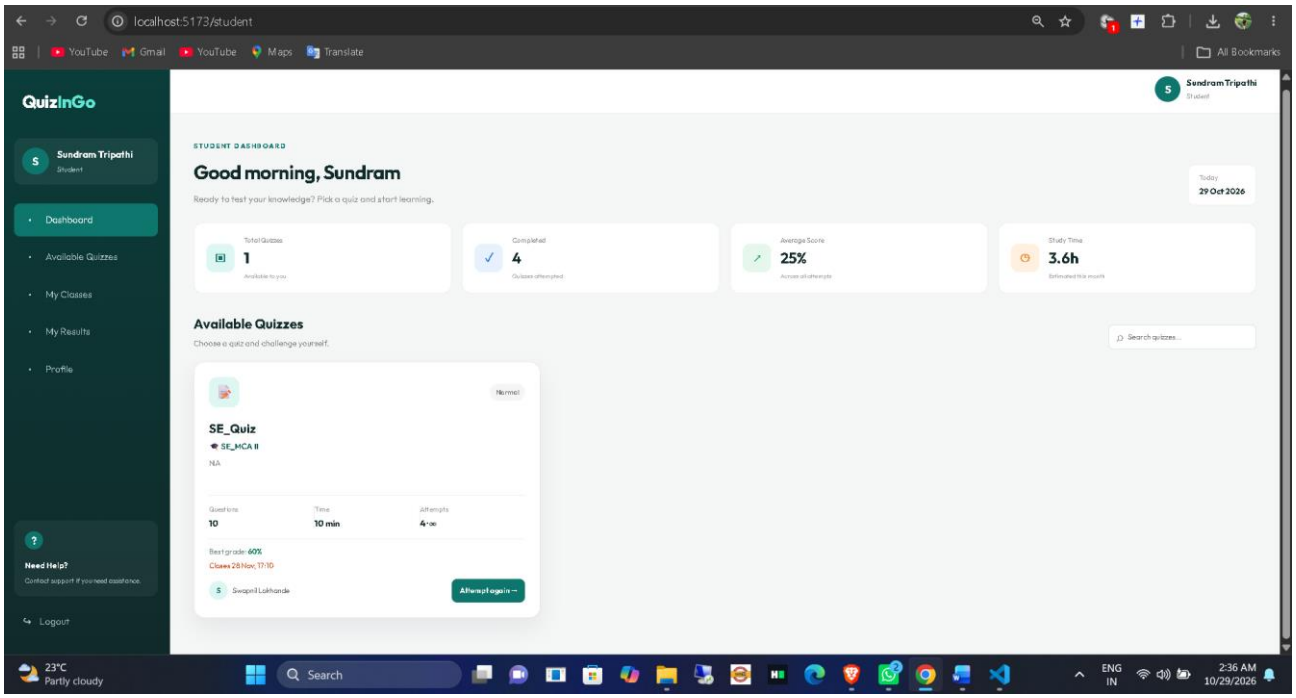
A.1 Login / Registration



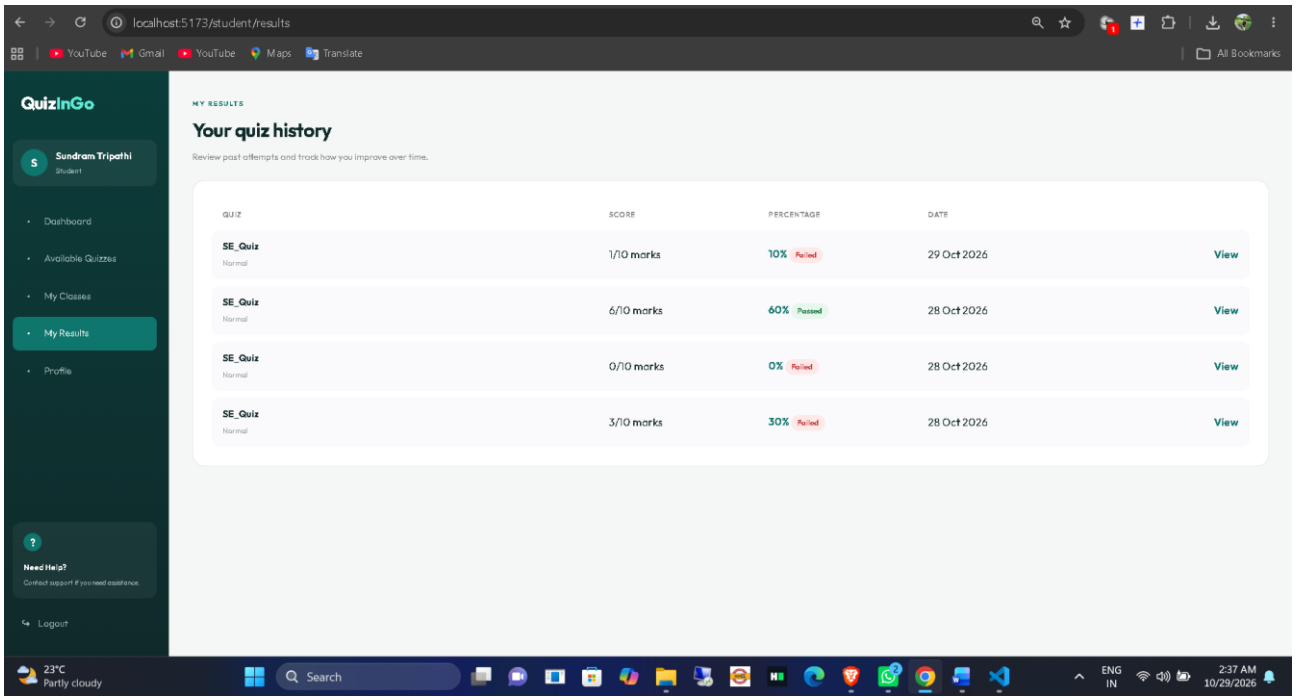
A.2 Teacher Dashboard / Quiz Creation



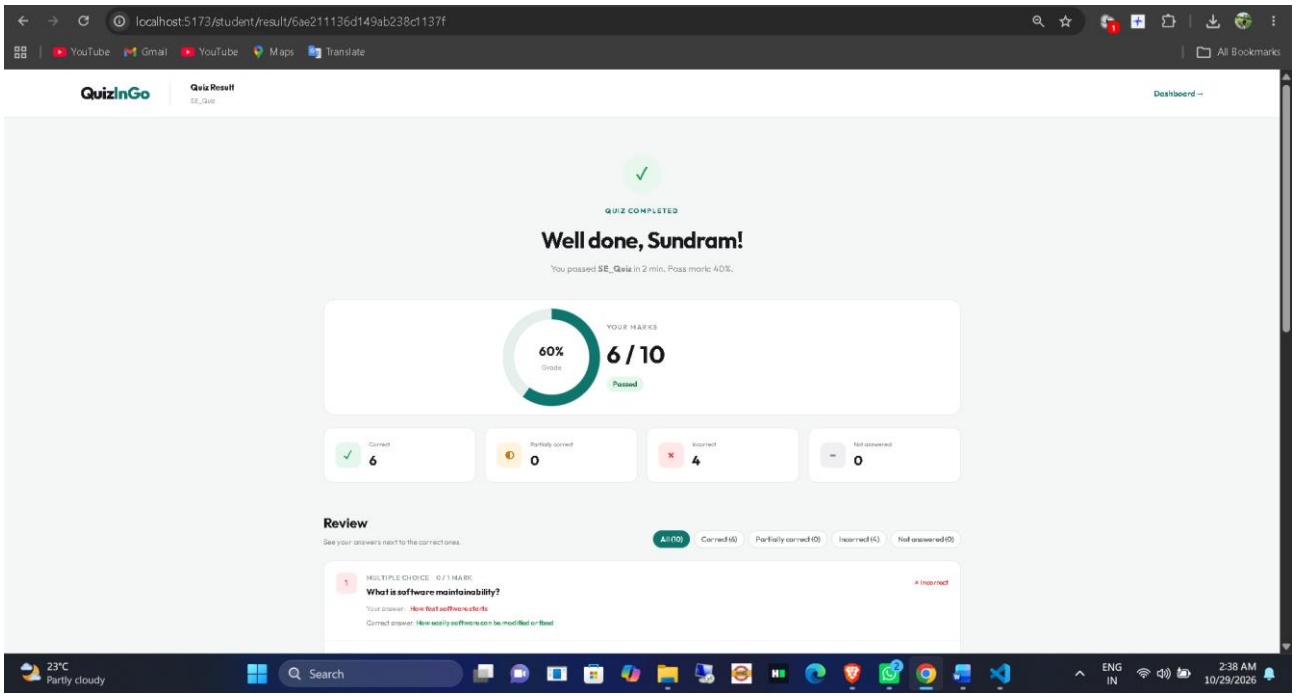
A.3 Student Quiz Attempt



A.4 Result / Feedback



A.5 Test Output



APPENDIX B: GLOSSARY

Term	Meaning in this project
SRS	Software Requirements Specification
DFD	Data Flow Diagram
UML	Unified Modeling Language
API	Application Programming Interface
JWT	JSON Web Token, if used in the project

REFERENCES

[1] CS545 Software Engineering lab handouts supplied by the course faculty. Add exact experiment title and date/version.

[2] QuizInGo project source code and README. Add repository/version details.

[3] Official documentation for technologies actually used. Add exact URLs and access date.